

ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА
МОСКВЫ

Государственное бюджетное профессиональное
образовательное учреждение города Москвы
«МОСКОВСКИЙ КОЛЛЕДЖ БИЗНЕС-ТЕХНОЛОГИЙ»
(ГБПОУ КБТ)

КУРСОВАЯ РАБОТА

по теме: «Проектирование и администрирование
серверной инфраструктуры веб-приложения»

СПЕЦИАЛЬНОСТЬ: 09.02.06 «Сетевое и системное
администрирование»

ГРУППА: СА31-19

ИСПОЛНИТЕЛЬ: _____

Чучканов Т. С.

РУКОВОДИТЕЛЬ: _____

Егоренков И. Ю.

Москва, 2022

СОДЕРЖАНИЕ

ВВЕДЕНИЕ

С ростом востребованности ресурсов, предоставляемых конечным пользователям через компьютерные сети, многие компании и отдельные группы людей или индивиды начинают создавать свои решения. Привлекает людей в таком подходе удобство для конечных пользователей и простота доступа из любой точки мира.

Для того чтобы иметь возможность обеспечивать доступ к таким ресурсам, необходимо соответствующее программное обеспечение.

Поскольку при таком подходе сервер, используемый для предоставления услуг, находится в общем доступе, он является привлекательной целью для злоумышленников.

Цель работы — показать на конкретном примере процесс выбора и конфигурации программного обеспечения, которое позволит развернуть сервис, являющийся веб-приложением, и сделать его доступным людям по всему миру, а также обеспечить безопасность как для самого сервера, так и для его клиентов.

Работа будет основана на сервисе, состоящем из двух серверных приложений: веб-сервера и приложения, циклично выполняющего определенный набор задач с одинаковыми временными промежутками.

Задачи:

- сравнить и выбрать технологии для развертывания серверных приложений;
- при необходимости использования ОС, рассмотреть возможные варианты распространенных серверных операционных систем и выбрать одну, подходящую для использования технологий, выбранных на предыдущем этапе;
- наглядно показать, каким образом можно выполнить настройку и защиту серверной инфраструктуры.

В первой части работы будут рассмотрены распространенные технологии и способы развертывания серверных приложений, а также часто применяемые серверные операционные системы.

Во второй части мы ознакомимся с компонентами веб-приложения для сервиса «Sufflain»¹. После этого, основываясь на компонентах используемого веб-приложения, будут выбраны наиболее подходящие технологии и ОС.

В третьей части будут предложены возможные варианты по улучшению текущего состояния выбора технологического стека для серверных частей веб-приложений.

1 <https://sufflain.icu/>

1 АНАЛИЗ СУЩЕСТВУЮЩИХ ТИПОВ ПЛАТФОРМ ДЛЯ РАЗВЕРТЫВАНИЯ ВЕБ-СЕРВИСОВ

1.1 Веб-сервисы как сущность

Перед проведением сравнения возможных типов платформ для развертывания веб-сервисов и прочего, необходимо сначала подробнее разобраться, что является веб-сервисом.

Веб-сервис (также часто употребляется выражение «веб-служба») – программная система, со стандартизированными интерфейсами, которая предоставляет свои услуги, используя технологии интернет, и идентифицируемая уникальным веб-адресом (URL-адресом).

Для обмена данными подобные сервисы зачастую используют специальные форматы для обмена (текстовыми) данными. Например, это может быть XML (или такие его варианты, как HTML и XHTML) или JSON.

Такой способ предоставления услуг делает возможным удобное взаимодействие при помощи устройства, способного подключаться к сети Интернет и имеющего возможность обрабатывать приведенные выше форматы обмена данными. Соответственно, почти любые современные устройства могут обеспечить возможность использовать подобные сервисы.

Стоит отметить, что веб-сайты и веб-приложения не являются веб-сервисами, так как веб-сервис не подразумевает прямое взаимодействие с конечным пользователем (при помощи удобного интерфейса, такого как графического). Не смотря на это, в большинстве случаев их развертывание имеет большие сходства, ведь используются одни и те же протоколы, например, HTTP. По этой причине и программное обеспечение, которое обеспечивает их работу, тоже зачастую применяется одно и то же.

Среди множества типов веб-сервисов можно выделить:

- XML-RPC;
- JSON-RPC;
- SOAP;
- REST.

XML- и JSON-RPC (где RPC расшифровывается как Remote Procedure Call и в переводе означает удаленный запуск процедур) — такой сервис предоставляет возможность другим компьютерам в общей сети вызывать определенные процедуры так, как это было бы при вызове обычных локальных процедур в программе.

SOAP (Simple Object Access Protocol) — спецификация протокола для обмена структурированной информацией в реализации веб-сервисов. Использует формат XML для обмена сообщениями и

полагается на использование протоколов уровня приложений; зачастую это HTTP.

REST (Representational state transfer) — стиль архитектуры программного обеспечения, который был создан для того чтобы направлять разработку архитектуры World Wide Web. REST — это набор ограничений того, как архитектура расширяемой распределенной гипермедиа системы должна себя вести.

1.2 Классификация платформ для развертывания веб-сервисов

Далее рассмотрим и сравним типы платформ применительно к веб-сервисам.

Возможные варианты можно классифицировать следующим образом:

- In-House
- Colocation
- Managed Hosting

In-House (часто In-House Hosting) — вероятно, наиболее старый способ развертывания программного обеспечения. Для него используется (серверное) оборудование, размещенное внутри самой компании, которая занимается разработкой ПО.

Colocation — способ, для которого компания пользуется услугами специального колокационного центра, внутри которого размещается необходимое оборудование, на котором в дальнейшем разворачивается программная часть.

Managed Hosting — данная категория может разделяться еще на несколько категорий: Dedicated Server, Virtual Private Server и Cloud Computing. Использование такого подхода заключается в аренде вычислительных ресурсов у провайдера. При этом, обслуживание

оборудования лежит на плечах администраторов провайдера такого рода услуг.

Dedicated Server — как и говорит название, это выделенный сервер, арендуемый у провайдера. Его ресурсы полностью в распоряжении лица, арендующего этот сервер. Обслуживание оборудования и обеспечение его бесперебойной работы производится сотрудниками провайдера, предоставляющего данную услугу. За дополнительную плату также можно заказать и администрирование программной части сервера.

Virtual Private Server (VPS) — сервер, являющийся виртуальной машиной, находящейся на сервере провайдера. При таком подходе ресурсы распределяются между виртуальными серверами всех клиентов и используют вычислительные мощности низлежащего сервера, к которому клиенты не имеют доступа.

Cloud Computing — доставка IT ресурсов «на лету» через Интернет, используя оплату по мере использования. Провайдеры облачных вычислений предоставляют вычислительную мощность, хранилище и базы данных по мере необходимости.

Существует несколько типов облачных вычислений. Одни из самых распространенных:

- Infrastructure as a Service (SaaS) — компоненты инфраструктуры, обычно располагаемые в локальных центрах обработки данных: серверы, хранилище, сеть, виртуализация.

- Platform as a Service (PaaS) — инструменты разработки, промежуточное ПО, операционные системы, управление базами данных, инфраструктура — все это обслуживается провайдером. Разработчики могут сфокусироваться на создании и развертывании ПО, не занимаясь администрированием инфраструктуры.
- Software as a Service (SaaS) — полноценный продукт, который управляется провайдером. В большинстве случаев — это приложения для конечных пользователей, такие как почтовые клиенты.
- Serverless computing — такая услуга позволяет разработчикам развертывать код программного обеспечения, не занимаясь настройкой и администрированием инфраструктуры. Не смотря на название, работа все равно обеспечивается за счет работы серверов.

Стоит отметить, что описанные выше типы платформ для развертывания ПО (а соответственно, и веб-сервисов) не являются полноценным перечислением таковых и их возможных категорий и подкатегорий.

1.3 Выбор типа платформы для развертывания веб-сервиса

Теперь, когда нам известно о возможных вариантах типов платформ, необходимо выбрать подходящий, а затем и конкретное решение.

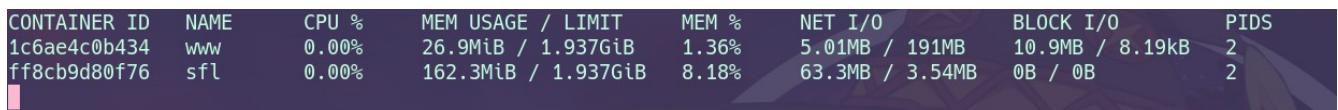
Сервис, который рассматривается в качестве примера, состоит из трех компонентов: серверное приложение, веб-сервер и база данных.

Известно, что серверное приложение и веб-сервер должны разворачиваться с использованием технологий контейнеризации, а именно, Docker. Это требование возникло по причине удобства для разработчика — все необходимое для работы программной части собрано в одном месте и позволяет легко разворачивать разные версии продукта в любой среде, поддерживающей контейнеризацию, без проблем с разрешением зависимостей; в дополнение к этому такой подход позволяет не «засорять» операционную систему сервера ненужными файлами и обеспечивает дополнительный уровень защиты благодаря изоляции.

Соответственно, необходимо выбрать способ развертывания сервиса, поддерживающий использование контейнеров Docker.

В таком случае остаются следующие варианты: In-House server, Colocation, Dedicated Server, VPS, Cloud Computing, IaaS, PaaS.

Исходя из проведенного мониторинга ресурсов, результат которого представлен на Рисунке 1, ни серверное приложение, ни веб-сервер не требуют больших вычислительных мощностей. Большую часть времени оба этих компонента находятся в режиме простоя. Серверное приложение выполняет определенный набор операций, включающих в себя взаимодействие с сетью, раз в полчаса, после чего снова переходит в режим простоя. Веб-сервер не обрабатывает большого количества данных, поэтому также не использует много ресурсов.



CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
1c6ae4c0b434	www	0.00%	26.9MiB / 1.937GiB	1.36%	5.01MB / 191MB	10.9MB / 8.19kB	2
ff8cb9d80f76	sfl	0.00%	162.3MiB / 1.937GiB	8.18%	63.3MB / 3.54MB	0B / 0B	2

Рисунок 1 - мониторинг использования системных ресурсов

Для работы сервиса используется база данных Realtime, предоставляемая сервисом Firebase. Это сервис, который помимо двух вариантов документоориентированных баз данных предлагает такие опции как аутентификация, отправку уведомлений и многое другое. Многие люди относят его в категорию BaaS (Backend as a Service). Соответственно, для работы с базой данных, используемой рассматриваемым нами сервисом, установка дополнительного ПО не требуется, ведь взаимодействие с БД происходит при помощи REST API.

Выбор будет осуществляться с учетом минимальных требований: 1 Гб оперативной памяти и до 1000 Гб сетевого трафика.

Итак, поскольку в данном случае нет нужды в высокопроизводительном оборудовании, такие опции как In-House

сервер, колокационные центры и выделенные серверы становятся явно неоптимальными решениями — это будут лишь ненужные дополнительные расходы. Сам сервис не рассчитан на широкую аудиторию пользователей, соответственно, расширение и сжатие ресурсов, не потребуются; компоненты, которые должны быть расположены на сервере, не связаны между собой напрямую, а их количество невелико. Поэтому использование Infrastructure as a Service тоже является неоптимальным вариантом.

Остаются три варианта: VPS, Cloud Computing и PaaS.

Рассмотрим варианты облачных вычислений от распространенных провайдеров.

Microsoft Azure — сервис облачных вычислений от крупной американской корпорации Microsoft. Среди большого числа услуг имеется возможность развертывания контейнеров. Согласно проведенным вычислениям стоимости, используя официальный калькулятор, была получена стоимость пользования сервисом за месяц. Итоговая стоимость равна чуть более \$41 USD/месяц. Ознакомиться со скриншотами стоимостей можно в Приложении.

Далее аналогичным образом был выполнен подсчет стоимости пользования Cloud Run от Google Cloud. Даже без указания большинства необходимых требований была получена сумма почти в \$120 USD/месяц.

Теперь ознакомимся с вариантом PaaS «Heroku» от от компании Salesforce.

Для развертывания контейнеров имеется бесплатный план, при котором активный сервис засыпает после 30 минут простоя. Вместе с этим, такой план не позволяет развертывать несколько разных типов сервисов. Стоимость самого дешевого плана начинается от \$25 USD/месяц.

Теперь ознакомимся с некоторыми распространенными вариантами VPS.

В качестве VPS провайдеров для сравнения были выбраны популярные российские AdminVPS и REG.RU.

VPS провайдер REG.RU имеет наиболее дешевый тариф с подходящими характеристиками за ~370 руб./месяц. Среди наиболее дешевых подходящих тарифов у AdminVPS есть за 809 руб./месяц (имеются и более дешевые).

Для подведения итога предлагаем обратиться к Таблице 1, содержащей в себе сравнение цен.

Таблица 1 - сравнение стоимости подходящих услуг для разных типов платформ

Провайдер	Тип рассматриваемой платформы	Стоимость подходящей услуги/месяц (руб.)
Microsoft Azure	Cloud Computing	~4 938
Google Cloud	Cloud Computing	~12 228
Heroku	PaaS	2 550
REG.RU	VPS	~370
AdminVPS	VPS	809

Таким образом, можно сделать вывод, что наиболее подходящим способом развертывания веб-сервиса по типу платформы для данного случая является именно VPS.

2 ПРАКТИЧЕСКОЕ РАЗВЕРТЫВАНИЕ ВЕБ-СЕРВИСА

2.1 Поиск подходящего ПО для выбранного типа платформы

После определения типа платформы, удовлетворяющего требованиям рассматриваемого нами сервиса, необходимо найти программное обеспечение, способное обеспечить стабильную работу программного продукта.

Начнем с главного компонента, движущего любую серверную инфраструктуру — операционной системы. На сегодняшний момент существует множество серверных ОС из разных семейств, и разрабатываемых для каких-то определенных наборов задач. Это могут быть системы из семейства BSD, такие как FreeBSD, OpenBSD, NetBSD и многие другие, либо HP-UX, IBM AIX, различные дистрибутивы Linux, системы Microsoft Windows, и так далее.

Выбирать систему, как и любой другой инструмент следует не исходя из «привычки» или удобства, а из применимости к каждому конкретному случаю использования. Для развертывания сервиса нам потребуется технология контейнеризации Docker, которая рассчитана на работу исключительно с ядром Linux. Безусловно, есть версии для Microsoft Windows и macOS, но даже они требуют установки виртуальных машин с дистрибутивами Linux. Если нет нужды в использовании таких ОС, то и не имеет смысла брать их исключительно для работы с Docker. Отсюда и становится ясно, что нам потребуется дистрибутив Linux.

Существует их большое множество, и каждый разрабатывается для каких-то определенных задач. Имеется два наиболее распространенных стабильных серверных дистрибутива: Debian GNU/Linux и Ubuntu. Последний основан на первом, но использует более свежие версии программного обеспечения, а сама система также обновляется гораздо

чаще, в отличие от первого. Вместе с этим, для Ubuntu имеется немного больше удобных инструментов, отчего работа становится приятнее, проще, и порой даже веселее. По этой причине выбран именно этот дистрибутив.

После выбора операционной системы перейдем к инструментам для обеспечения защиты: межсетевым экранам и IPS (Intrusion Prevention System).

Может возникнуть вопрос — почему? Как минимум по той причине, что по всему миру находятся боты, регулярно осуществляющие поиск уязвимых устройств, доступных по сети. Делают они это при помощи сканирования портов, попыток удаленного запуска кода, например, при помощи интерпретатора языка PHP, получения переменных среды, подбора паролей и так далее. Поскольку настраиваемый сервис должен бесперебойно работать, и обслуживать пользователей, не подвергая их и себя опасности, его следует защитить

Начнем с фаерволла. Для UNIX-подобных операционных систем существует множество качественных утилит такого типа. Все они нацелены на разные потребности и уровни знаний пользователей. Среди самых популярных можно выделить: iptables, firewallld, Uncomplicated Firewall (сокр. ufw). Сразу стоит отметить, что для работы серверной инфраструктуры сервиса, для которого нам необходимо выполнить настройку, не требуется межсетевого экрана, имеющего гибкий функционал и большое количество функций. Как раз для такой ситуации хорошо подходит ufw. Безусловно, данная утилита способная осуществлять такие вещи, как port forwarding, но делается

это при помощи конфигурационных файлов. Команды для настройки функционала, который потребуется нам в дальнейшем проще и короче таковых у iptables и firewallld. Соответственно, имеет смысл использовать инструмент, который позволит настраивать инфраструктуру быстрее и с большим комфортом.

Далее потребуется не менее важный компонент — система предотвращения доступа (не аппаратная). Так как сервер будет доступен через Интернет, он автоматически станет целью для злоумышленников.

Программного обеспечения такого типа также имеется немало: например, fail2ban, Snort, Tripwire, Suricata, и так далее. Одина из самых популярных IDS утилит среди владельцев VPS серверов fail2ban. Она проста в использовании, имеет хорошую конфигурацию по умолчанию, которая защитит сервер в большинстве сценариев. По этой причине многим пользователям достаточно лишь установить данную программу и запустить сервис.

Без специальных инструментов удаленного доступа невозможно администрировать VPS сервер, что логично, ведь он находится за пределами сети пользователя. Обычно для доступа к серверам используется SSH или Telnet. Оба протокола позволяют устанавливать терминальные соединения через сеть. В современном мире люди стараются отказываться от Telnet в пользу SSH, так как последний шифрует весь трафик, передаваемый между клиентом и сервером. На большинстве UNIX-подобных систем, как и в нашем случае, уже установлен пакет OpenSSH.

Поскольку даже в наше время соединение не всегда может быть стабильным и быстрым, для плавности и удобства работы можно прибегнуть к использованию Mosh. Эта программа рассчитана на работу с rlogin, а также протоколами SSH и Telnet в сетях с неустойчивым подключением. Сессия не прерывается даже после переподключения к сети. Достигается это за счет использования протокола SSP (State Synchronization Protocol), рассмотрение которого выходит за рамки данного проекта.

2.2 Установка и конфигурация ПО

Теперь, когда нами успешно был произведен выбор необходимого программного обеспечения, перейдем к демонстрации его настройки. Как и было сказано ранее, все конфигурации будут показаны на примере сервера сервиса «Sufflain».

Настройка будет включать в себя установку самого программного обеспечения, и осуществляться в следующем порядке:

1. Настройка SSH;
2. Установка Mosh;
3. Установка и настройка межсетевого экрана;
4. Установка системы предотвращения вторжений;
5. Установка Docker и развертывание контейнеров.

2.2.1 Подготовка к установке и настройке

Перед тем, как приступить к непосредственной настройке выбранных программ, необходимо немного подготовить систему.

Выполнять в дальнейшем удаленные подключения для взаимодействия с сервером при помощи root аккаунта не является хорошей идеей, так как это может поставить под угрозу безопасность сервера. Мы создадим обычного пользователя, не обладающего повышенными привилегиями, и разрешим SSH доступ только для него. При этом придется устранить один недостаток.

Свежеустановленная система Ubuntu имеет специальную утилиту под названием `sudo`, которая позволяет выполнять действия от лица другого пользователя. Возможен такой сценарий, что при успешном получении удаленного SSH доступа к серверу (например, при Bruteforce атаке) злоумышленник попытается выполнить какие-либо действия,

требующие прав root пользователя. В такой ситуации sudo сыграет такому индивиду на руку.

Чтобы этого избежать, мы удалим данную программу. Соответственно, в случае необходимости администрирования, сценарий получения удаленного доступа администратора будет выглядеть следующим образом:

1. Создание удаленного сеанса для непривилегированного пользователя;
2. Вход в root аккаунт при помощи «su -l»

Теперь продемонстрируем на практике, как произвести описанные выше манипуляции над аккаунтами.

Для выполнения входа в root аккаунт необходимо ввести команду, представленную на Рисунке 2.

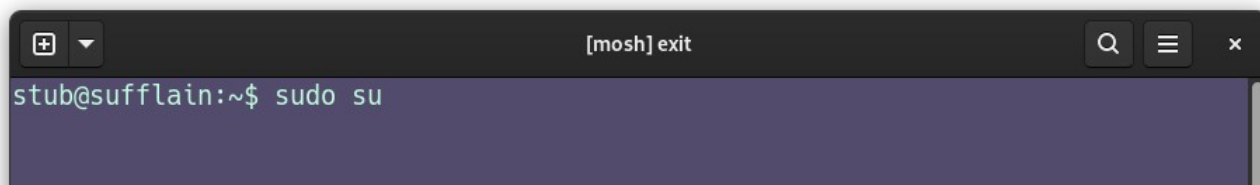


Рисунок 2 - Вход в root аккаунт

При создании пользователя следует указать флаг «-m» для автоматического создания домашней директории пользователя и опцию «-d» для указания пути размещения домашней директории пользователя. Использование команды для такой операции и отображение информации о созданной директории пользователя показаны на Рисунке 3. Имя созданного пользователя было закрашено в целях обеспечения безопасности сервера.

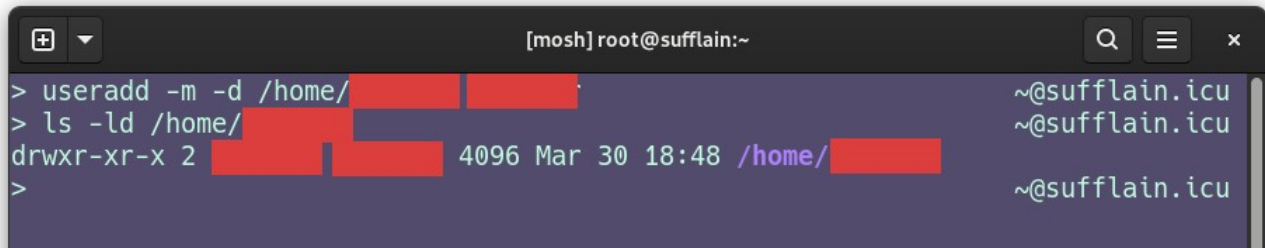


Рисунок 3 - Создание пользователя

Полное удаление утилиты `sudo`, включая все ее конфигурационные файлы показано на Рисунке 4.

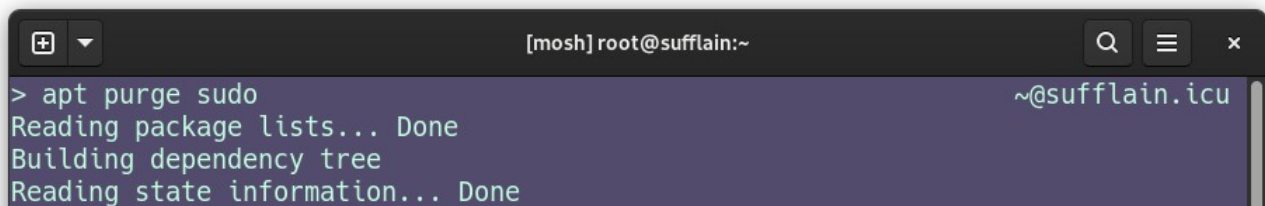


Рисунок 4 - Полное удаление sudo, включая конфигурационные файлы

После произведенных операций также необходимо установить пароль root пользователя, чтобы иметь доступ к этому аккаунту. Исходя из соображений безопасности пароль должен быть сложным.

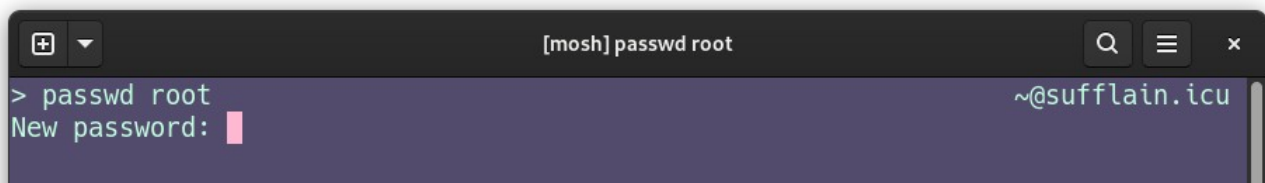


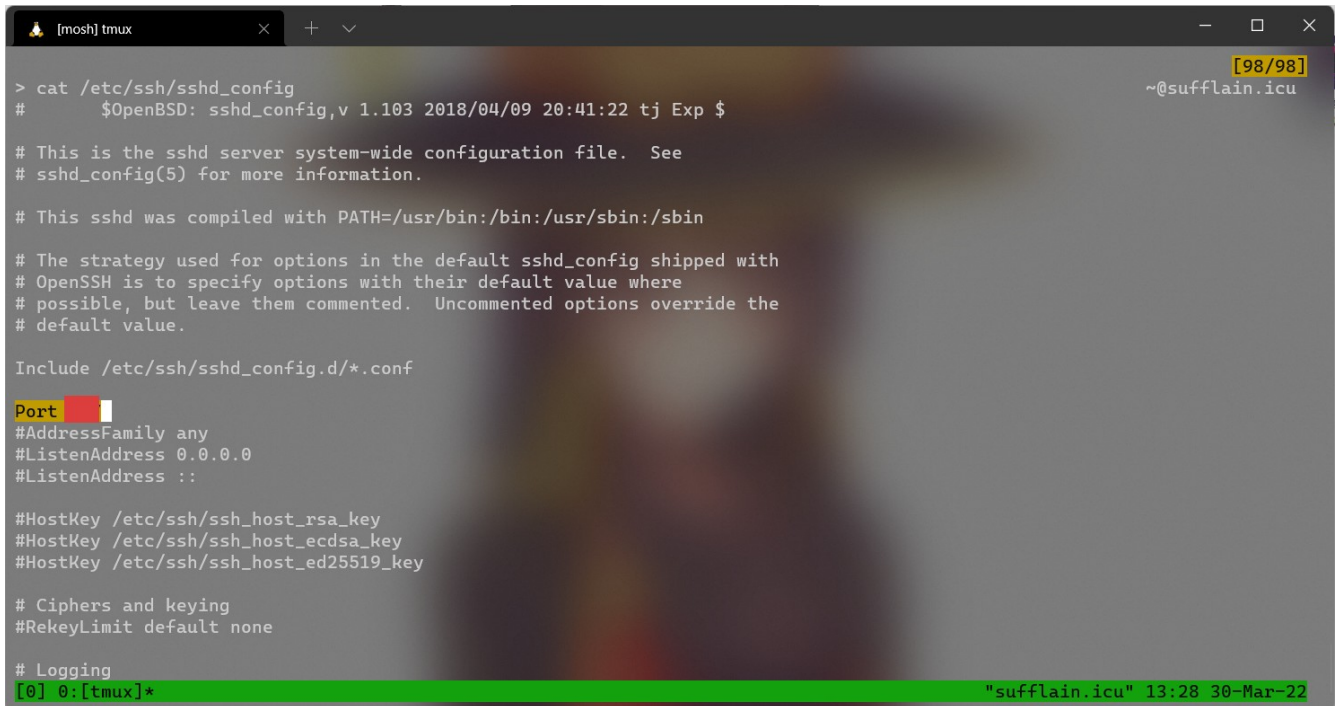
Рисунок 5 - Смена пароля пользователя root

Программа «passwd» попросит дважды ввести новый пароль.

2.2.2 Настройка SSH

Многие скрипты для bruteforce атак используют стандартный порт SSH. Для уменьшения вероятности проведения подобной атаки изменим порт SSH. В целях безопасности, имя пользователя и номер порта были закрашены. Вся конфигурация SSH осуществляется в файле «/etc/ssh/sshd_config».

Изменение номера порта SSH, отключение доступа при помощи root, а также разрешение доступа при помощи созданного нами ранее пользователя представлены на Рисунках 6, 7 и 8 соответственно.



```
[mosh] tmux
> cat /etc/ssh/sshd_config
#
#OpenBSD: sshd_config,v 1.103 2018/04/09 20:41:22 tj Exp $

# This is the sshd server system-wide configuration file.  See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/bin:/bin:/usr/sbin:/sbin

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented.  Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
[0] 0:[tmux]*
"sufflain.icu" 13:28 30-Mar-22
```

Рисунок 6 - Изменение номера порта SSH

```

[mosh] tmux
(2/2 results) [85/98]

Include /etc/ssh/sshd_config.d/*.conf

Port 
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

(search down) PermitRootLogin

```

Рисунок 7 - Отключение возможности использования root аккаунта для подключения при помощи SSH

```

[mosh] tmux
[0/98]

#TCPKeepAlive yes
#PermitUserEnvironment no
#Compression delayed
#ClientAliveInterval 0
#ClientAliveCountMax 3
#UseDNS no
#PidFile /var/run/sshd.pid
#MaxStartups 10:30:100
#PermitTunnel no
#ChrootDirectory none
#VersionAddendum none

# no default banner path
Banner /etc/issue.net

# Allow client to pass locale environment variables
AcceptEnv LANG LC_*

# override default of no subsystems
Subsystem sftp /usr/lib/openssh/sftp-server

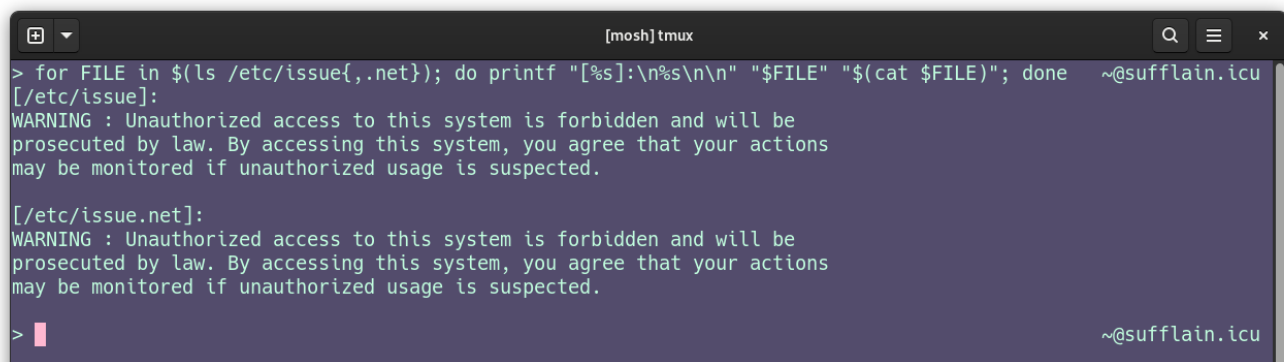
# Example of overriding settings on a per-user basis
#Match User anoncvs
#    X11Forwarding no
#    AllowTcpForwarding no
#    PermitTTY no
#    ForceCommand cvs server
AllowUsers root
>
[0] 0:[tmux]*
~@sufflain.icu
"sufflain.icu" 13:29 30-Mar-22

```

Рисунок 8 - Разрешение использования созданного ранее пользователя для входа через SSH

Хорошей практикой является установка баннеров дня, отображаемых при входе в систему, и предупреждающих о последствиях несанкционированного досутпа. Для этого в конфигурационный файл SSH добавляется «Banner» + путь к файлу баннера, как показано на Рисунке 8.

Файл «/etc/issue.net» используется для удаленных подключений, а файл «/etc/issue» для локальных. Пример содержимого этих файлов показан на Рисунке 9.



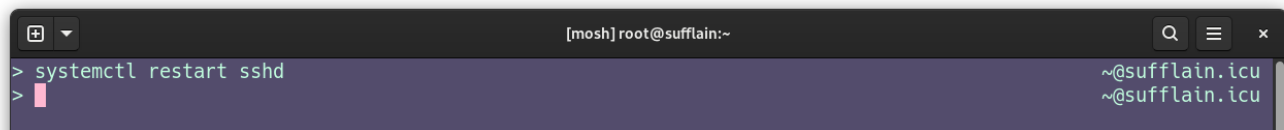
```
[mosh] tmux
> for FILE in $(ls /etc/issue{,.net}); do printf "[%s]:\n%s\n\n" "$FILE" "$(cat $FILE)"; done ~@sufflain.icu
[/etc/issue]:
WARNING : Unauthorized access to this system is forbidden and will be
prosecuted by law. By accessing this system, you agree that your actions
may be monitored if unauthorized usage is suspected.

[/etc/issue.net]:
WARNING : Unauthorized access to this system is forbidden and will be
prosecuted by law. By accessing this system, you agree that your actions
may be monitored if unauthorized usage is suspected.

> ~@sufflain.icu
```

Рисунок 9 - Содержимое файлов с текстами для баннеров дня

Настройка демона SSH, как и почти любого другого сервиса завершается его перезапуском для применения изменений. Пример показан на Рисунке 10.

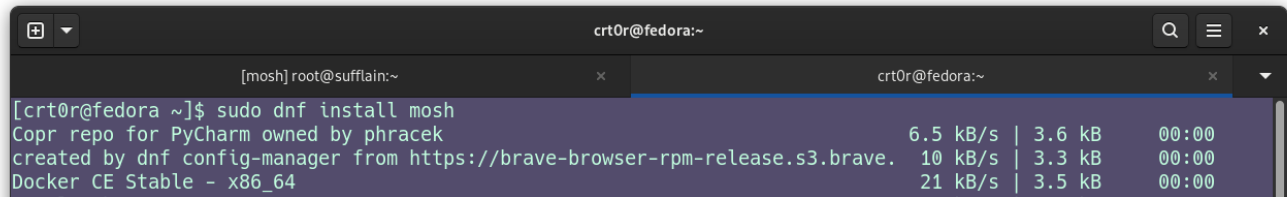


```
[mosh] root@sufflain:~
> systemctl restart sshd ~@sufflain.icu
> ~@sufflain.icu
```

Рисунок 10 - Перезапуск сервиса SSH

2.2.3 Установка Mosh

Для использования Mosh не требуется производить конфигурацию и запускать сервис, так как его нет. Достаточно установить программу на клиент и на сервер. Остальное работает при помощи уже существующих протоколов удаленного доступа. Пример установки на клиент и сервер показан на рисунках 11 и 12 соответственно.

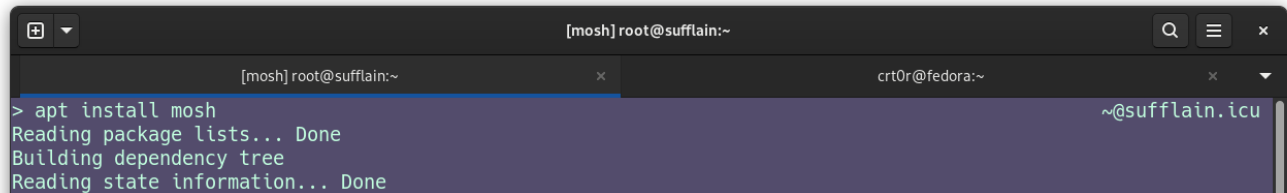


```

crt0r@fedora:~
[mosh] root@sufflain:~
[crt0r@fedora ~]$ sudo dnf install mosh
Copr repo for PyCharm owned by phracek
created by dnf config-manager from https://brave-browser-rpm-release.s3.brave.
6.5 kB/s | 3.6 kB 00:00
10 kB/s | 3.3 kB 00:00
21 kB/s | 3.5 kB 00:00

```

Рисунок 11 - Установка Mosh на клиентский компьютер (Fedora)



```

[mosh] root@sufflain:~
> apt install mosh
Reading package lists... Done
Building dependency tree
Reading state information... Done

```

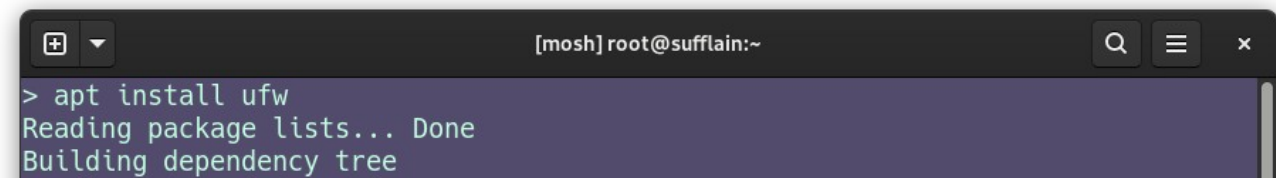
Рисунок 12 - Установка Mosh на сервер (Ubuntu)

Для использования нужно указать протокол, при необходимости порт протокола, и порт для работы самого Mosh. Пример использования будет показан далее, после настройки межсетевого экрана.

2.2.4 Установка и настройка межсетевого экрана

Теперь перейдем к конфигурации основного компонента, обеспечивающего защиту сервера.

Установка выполняется при помощи системного пакетного менеджера, как и с остальными программами в нашем случае.



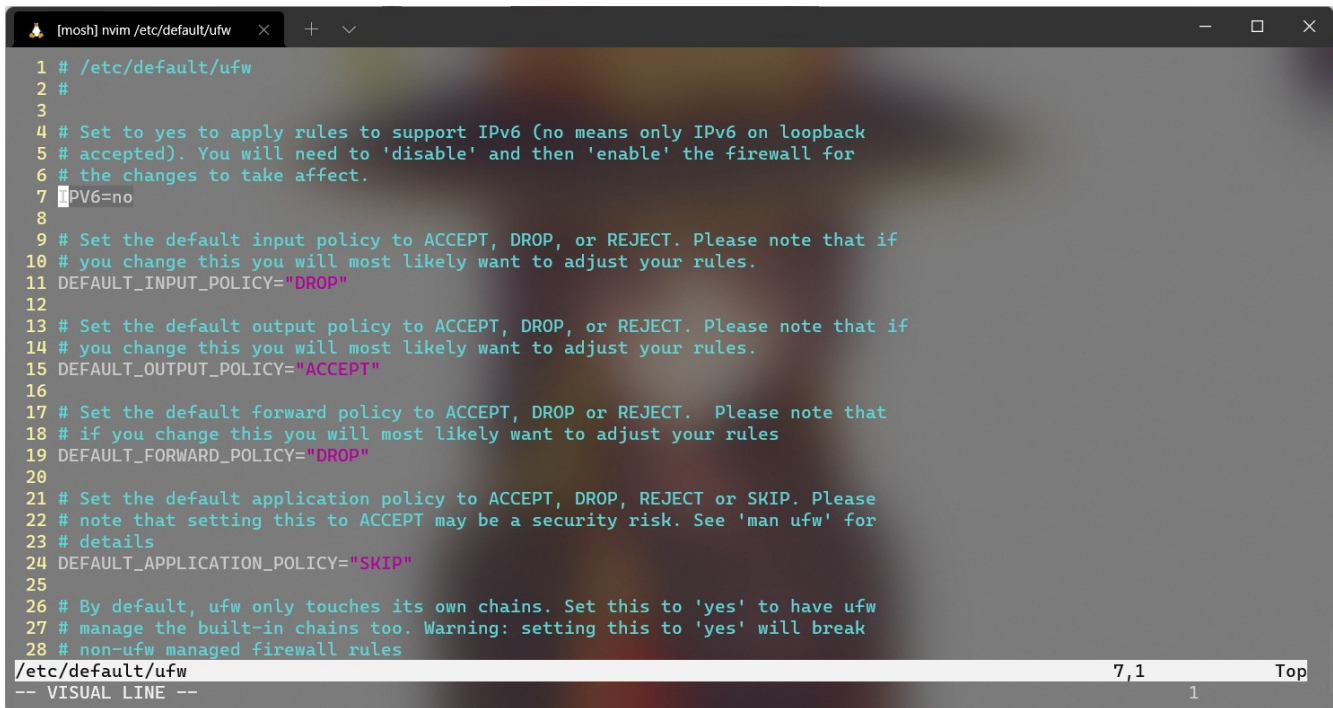
```

[mosh] root@sufflain:~
> apt install ufw
Reading package lists... Done
Building dependency tree

```

Рисунок 13 - Установка фаервола ufw

Поскольку сервер использует только IP-адрес Ipv4, следует отключить Ipv6 в конфигурационном файле ufw, находящемся по пути /etc/default/ufw.



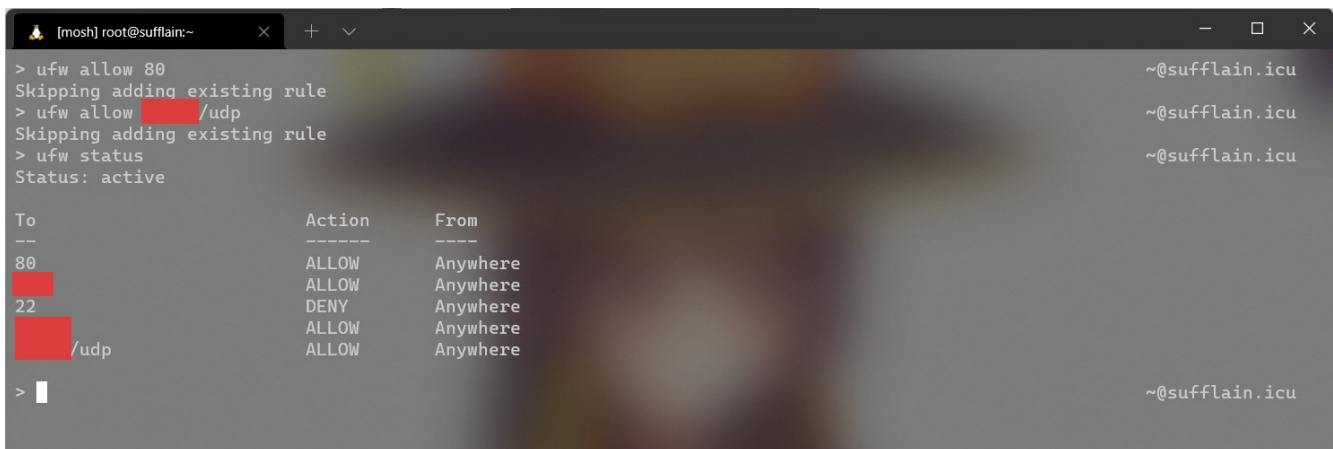
```

1 # /etc/default/ufw
2 #
3
4 # Set to yes to apply rules to support IPv6 (no means only IPv6 on loopback
5 # accepted). You will need to 'disable' and then 'enable' the firewall for
6 # the changes to take affect.
7 IPV6=no
8
9 # Set the default input policy to ACCEPT, DROP, or REJECT. Please note that if
10 # you change this you will most likely want to adjust your rules.
11 DEFAULT_INPUT_POLICY="DROP"
12
13 # Set the default output policy to ACCEPT, DROP, or REJECT. Please note that if
14 # you change this you will most likely want to adjust your rules.
15 DEFAULT_OUTPUT_POLICY="ACCEPT"
16
17 # Set the default forward policy to ACCEPT, DROP or REJECT. Please note that
18 # if you change this you will most likely want to adjust your rules
19 DEFAULT_FORWARD_POLICY="DROP"
20
21 # Set the default application policy to ACCEPT, DROP, REJECT or SKIP. Please
22 # note that setting this to ACCEPT may be a security risk. See 'man ufw' for
23 # details
24 DEFAULT_APPLICATION_POLICY="SKIP"
25
26 # By default, ufw only touches its own chains. Set this to 'yes' to have ufw
27 # manage the built-in chains too. Warning: setting this to 'yes' will break
28 # non-ufw managed firewall rules
/etc/default/ufw
-- VISUAL LINE --

```

Рисунок 14 - Выключение Ipv6 в конфигурационном файле ufw

Далее требуется открыть порты, которые будут использоваться для входящих подключений: на данный момент это 80, а также один TCP порт для работы SSH и один UDP порт для работы Mosh.



```

> ufw allow 80
Skipping adding existing rule
> ufw allow /udp
Skipping adding existing rule
> ufw status
Status: active

```

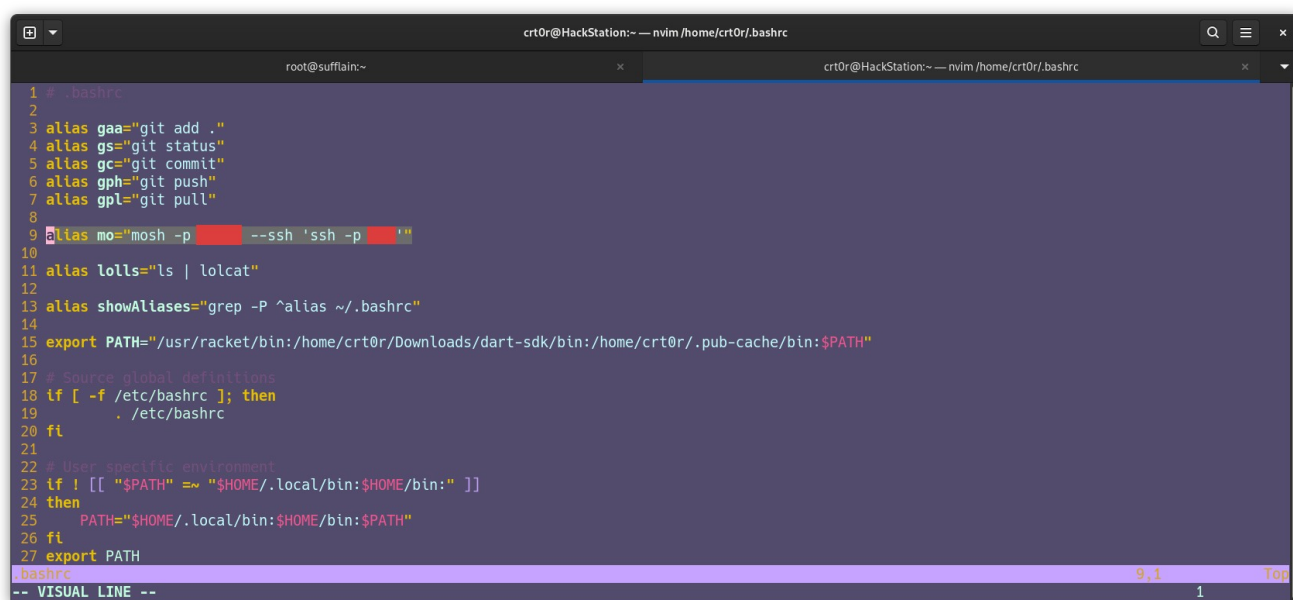
To	Action	From
80	ALLOW	Anywhere
22	DENY	Anywhere
/udp	ALLOW	Anywhere

Рисунок 15 - Открытие требуемых портов при помощи ufw

Стоит обратить внимание, на то, что для манипуляций над UDP портами, после номера порта добавляется «/udp» без кавычек.

После применения нужных конфигураций, для их применения, требуется выполнить перезапуск сервиса. Для этого используется команда, показанная на Рисунке 10. Название сервиса соответствует названию самой программы - «ufw».

С используемой нами конфигурацией, для использования Mosh, требуется указание множества опций, поэтому для удобства можно создать alias командной оболочки, как показано на Рисунке ниже.



```

1 # .bashrc
2
3 alias gaa="git add ."
4 alias gs="git status"
5 alias gc="git commit"
6 alias gph="git push"
7 alias gpl="git pull"
8
9 alias mo="mosh -p --ssh 'ssh -p'"
10
11 alias lolls="ls | lolcat"
12
13 alias showAliases="grep -P ^alias ~/.bashrc"
14
15 export PATH="/usr/racket/bin:/home/crt0r/Downloads/dart-sdk/bin:/home/crt0r/.pub-cache/bin:$PATH"
16
17 # Source global definitions
18 if [ -f /etc/bashrc ]; then
19     . /etc/bashrc
20 fi
21
22 # User specific environment
23 if ! [ "$PATH" = "$HOME/.local/bin:$HOME/bin:" ]
24 then
25     PATH="$HOME/.local/bin:$HOME/bin:$PATH"
26 fi
27 export PATH
28
29 .bashrc
-- VISUAL LINE --

```

Рисунок 16 - Создание alias для Mosh

После этого, вместо длинного «mosh -p <порт> --ssh 'ssh -p <порт>' <пользователь>@<хост>» можно применять «mo <пользователь>@<хост>».

2.2.5 Установка системы предотвращения вторжения

Как и было сказано ранее, в большинстве случаев, включая наш, достаточно конфигурации по умолчанию, поэтому достаточно ограничиться установкой и запуском сервиса.

Установка выполняется таким же способом, как и показано на Рисунке 13. В качестве названия пакета используется «fail2ban».

После запуска сервиса fail2ban при помощи «systemctl enable —now fail2ban» можно проверить, успешно ли запустился сервис, используя «systemctl status fail2ban». Будет получен результат, похожий на следующий. Подобным образом следует проверять все запускаемые сервисы.

```

systemctl status fail2ban

● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/lib/systemd/system/fail2ban.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2022-03-12 08:38:02 MSK; 3 weeks 0 days ago
     Docs: man:fail2ban(1)
   Process: 450737 ExecReload=/usr/bin/fail2ban-client reload (code=exited, status=0/SUCCESS)
    Main PID: 506 (f2b/server)
       Tasks: 5 (limit: 2311)
      Memory: 15.6M
     CGroup: /system.slice/fail2ban.service
            └─506 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

Mar 12 08:38:02 sufflaine.icu systemd[1]: Starting Fail2Ban Service...
Mar 12 08:38:02 sufflaine.icu systemd[1]: Started Fail2Ban Service.
Mar 12 08:38:03 sufflaine.icu fail2ban-server[506]: Server ready
Mar 29 18:53:07 sufflaine.icu systemd[1]: Reloading Fail2Ban Service.
Mar 29 18:53:07 sufflaine.icu fail2ban-client[450737]: OK
Mar 29 18:53:07 sufflaine.icu systemd[1]: Reloaded Fail2Ban Service.
~
~
~
lines 1-17/17 (END)

```

Рисунок 17 - Проверка статуса работы fail2ban

2.2.6 Установка и настройка Docker

Для успешной установки Docker на имеющуюся систему, следует обратиться к официальной документации по установке, находящейся по адресу «<https://docs.docker.com/engine/install/>». На данной странице, как показано на Рисунке 18, можно найти список ссылок с инструкциями для разных операционных систем. Мы воспользуемся ссылкой для систем Ubuntu.

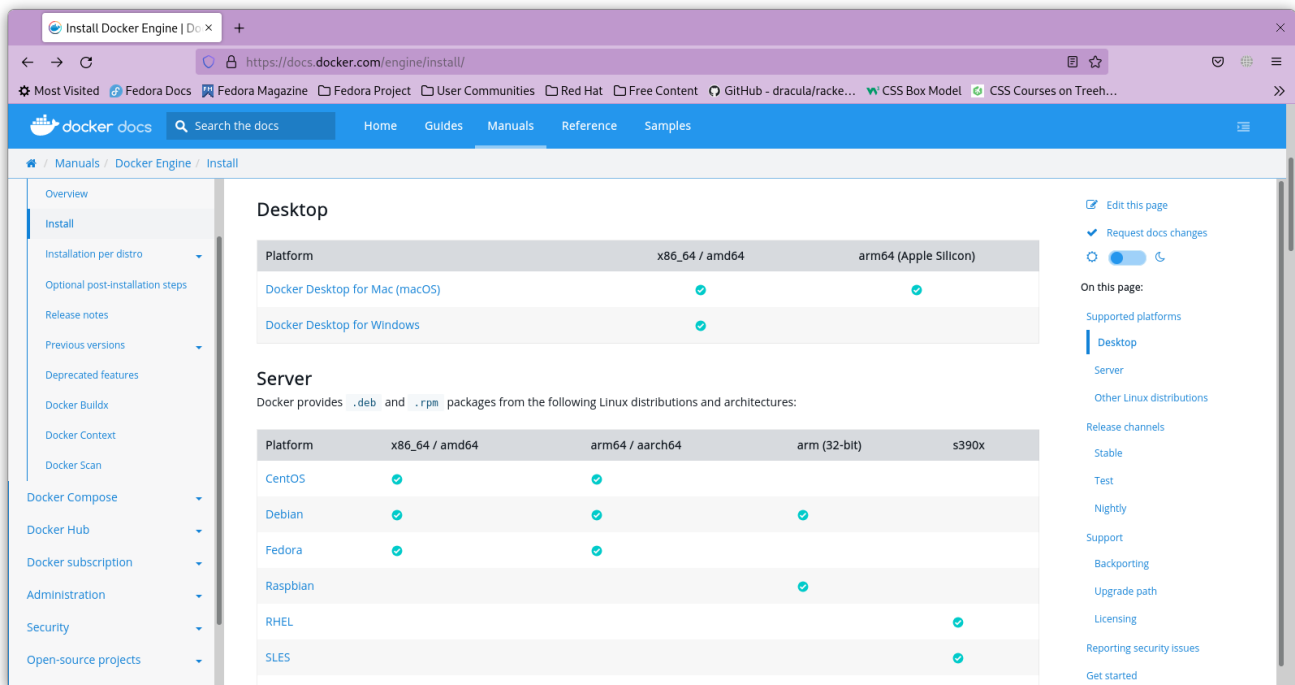


Рисунок 18 - Раздел с инструкциями по установке Docker из официальной документации

Согласно нескольким пунктам, показанных на Рисунках 19 и 20, для систем Ubuntu сначала требуется добавить репозиторий Docker, после чего можно будет приступить к установке.

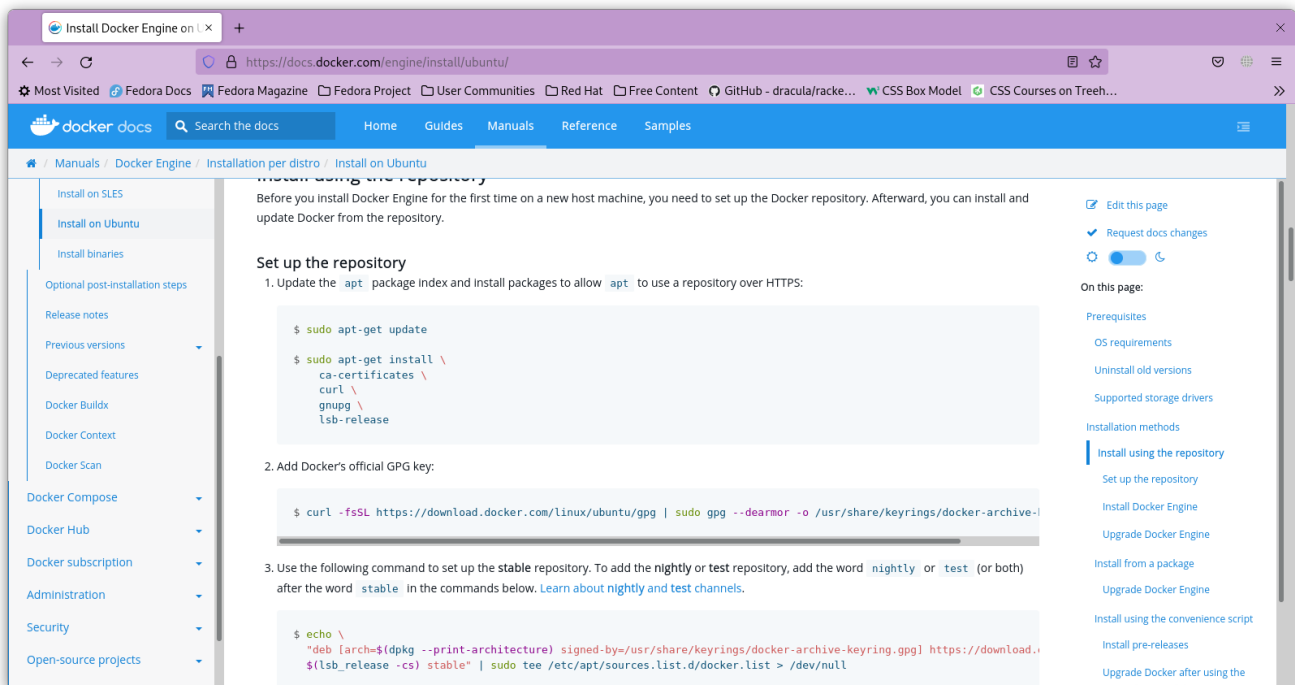


Рисунок 19 - Инструкции по добавлению репозитория Docker

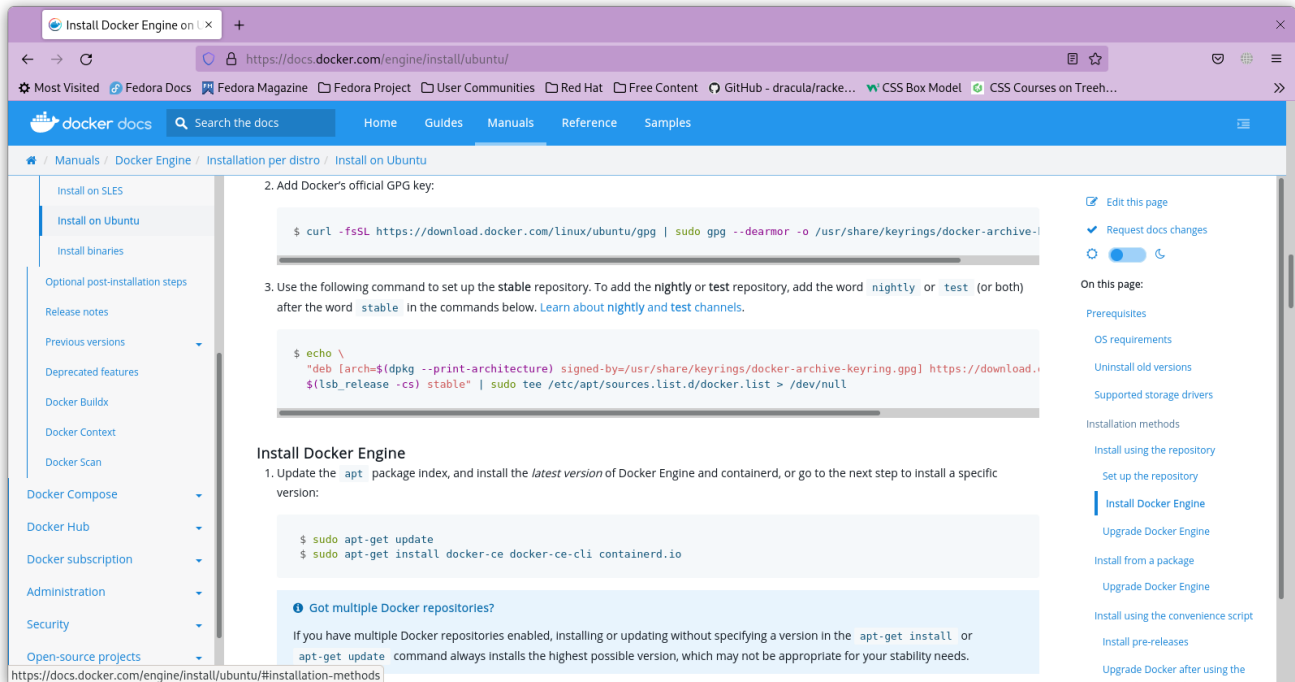
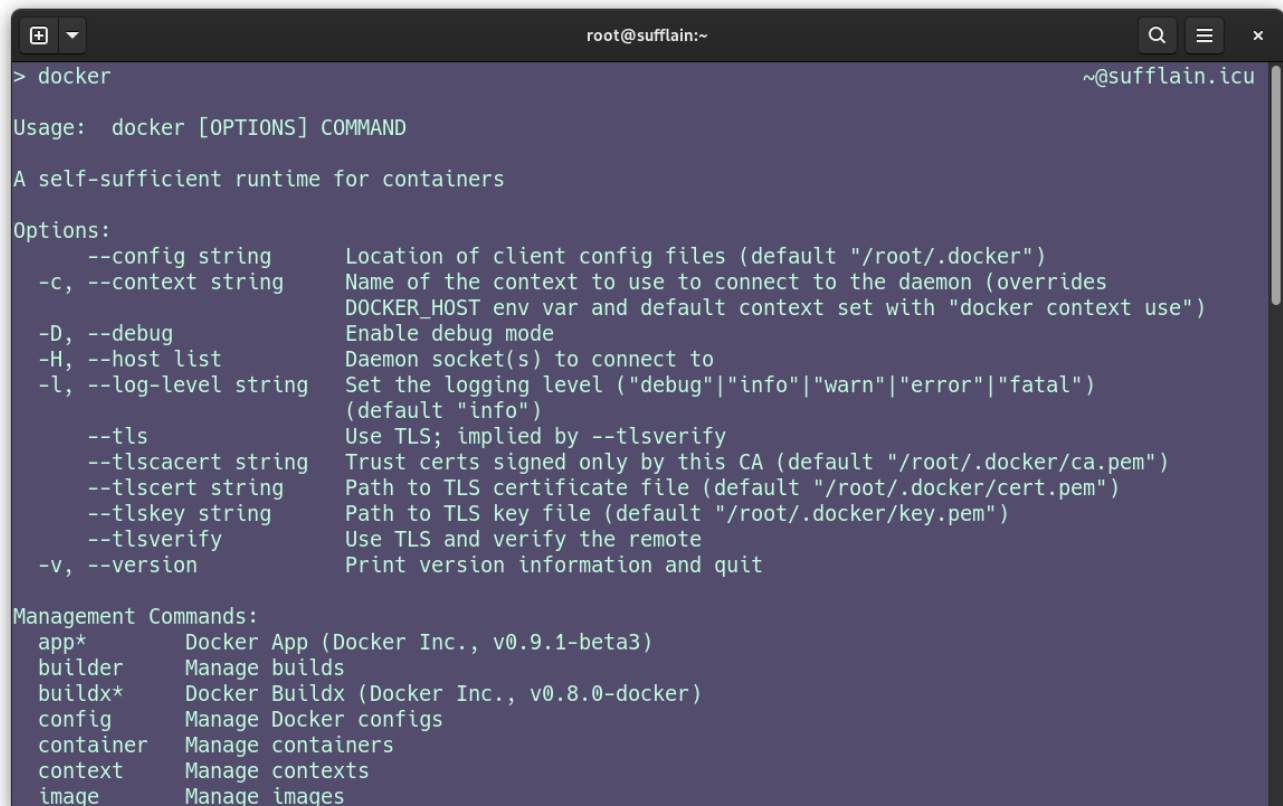


Рисунок 20 - Инструкции по установке Docker

Приводить дубликат показанных выше команд после ввода в терминал не имеет смысла, поэтому этот этап будет пропущен.

Проверить успешность установки Docker можно введя соответствующую команду, как показано на Рисунке 21.



```
> docker

Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

Options:
  --config string      Location of client config files (default "/root/.docker")
  -c, --context string  Name of the context to use to connect to the daemon (overrides
                        DOCKER_HOST env var and default context set with "docker context use")
  -D, --debug           Enable debug mode
  -H, --host list       Daemon socket(s) to connect to
  -l, --log-level string Set the logging level ("debug"|"info"|"warn"|"error"|"fatal")
                        (default "info")
  --tls                Use TLS; implied by --tlsverify
  --tlscacert string    Trust certs signed only by this CA (default "/root/.docker/ca.pem")
  --tlscert string       Path to TLS certificate file (default "/root/.docker/cert.pem")
  --tlskey string        Path to TLS key file (default "/root/.docker/key.pem")
  --tlsverify           Use TLS and verify the remote
  -v, --version         Print version information and quit

Management Commands:
  app*      Docker App (Docker Inc., v0.9.1-beta3)
  builder   Manage builds
  buildx*   Docker Buildx (Docker Inc., v0.8.0-docker)
  config    Manage Docker configs
  container Manage containers
  context   Manage contexts
  image     Manage images
```

Рисунок 21 - Проверка установки Docker

В случае успеха, вывод данной команды будет выглядеть примерно так, как отображено выше.

После установки можно приступить к развертыванию контейнеров с сервисами.

Это можно сделать множеством способов. Для личного удобства выбран следующий процесс получения образов и их использования:

- Сохранение образа в архив
- Выгрузка образа на сервер при помощи scp
- Загрузка образа из архива
- Запуск контейнера

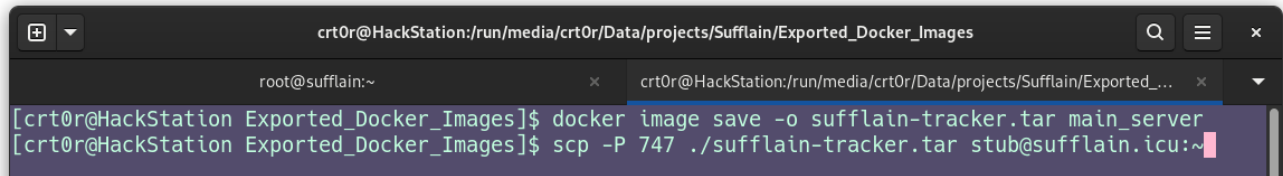


Рисунок 22 - Сохранение образа и его выгрузка на сервер

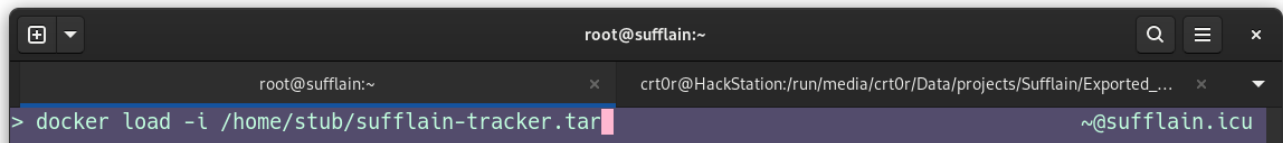


Рисунок 23 - Загрузка образа из архива

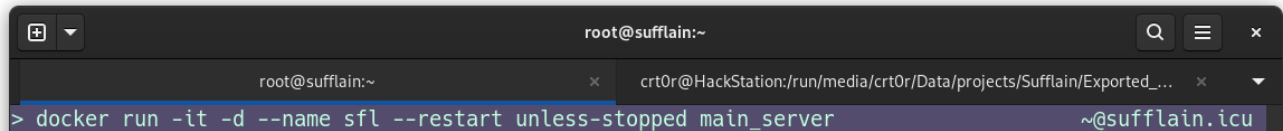


Рисунок 24 - Запуск приложения, не имеющего доступа к нему извне

При запуске первого контейнера использовались флаги «-it» и «-d» для того чтобы логи приложения из стандартного вывода сохранялись сервисом Docker, и могли далее быть запрошены командой «docker logs <имя_контейнера>». Опция «--restart unless-stopped» заставляет докер запускать контейнер заново в случае его падения, либо после перезапуска системы или сервиса.

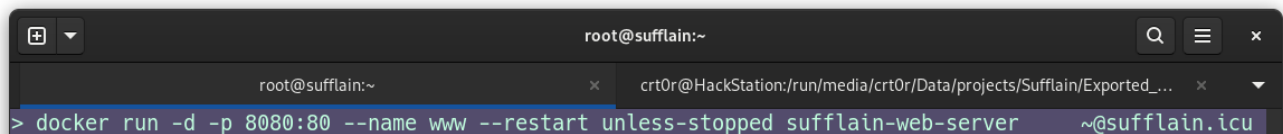


Рисунок 25 - Запуск приложения, доступ к которому необходим по определенному порту

Опция «-p [порт:]порт» привязывает порт хоста к порту контейнера. Таким образом можно получать доступ, например, к веб-

серверам, развернутым внутри контейнеров, при обращении к самому хосту так, будто сервис запущен напрямую в системе.

Также в обоих случаях использовалась опция «--name <имя>» для привязки имени к контейнеру. В дальнейшем, контейнером можно будет управлять, обращаясь к нему по этому имени вместо случайно сгенерированного идентификатора, что гораздо удобнее и быстрее.

Удостовериться в работе контейнеров можно, как показано на Рисунке 26. Если по какой-либо причине контейнер не отображается в выводе этой команды, он «упал» – завершился с ошибкой. Посмотреть все контейнеры, включая остановленные, можно запросив команду «docker ps -a».

```

> docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS                               NAMES
1c6ae4c0b434   sufflain-web-server  "/build/sflw -a0.0.0..."  3 weeks ago   Up 3 weeks   0.0.0.0:8080->80/tcp, :::8080->80/tcp  www
ff8cb9d80f76   main_server        "/root/app/bin/sfl -..."    4 weeks ago   Up 5 days                      sfl
  
```

Рисунок 26 - Отображение запущенных контейнеров